

In `useEffect`, If I pass an empty array as the dependency, then It will make the api call only once right?

**Short answer:** Yes — but with an important nuance.

In React, when you write:

```
jsx
```

[Copy code](#)

```
useEffect(() => {  
  // API call  
}, []);
```

The empty dependency array ( `[]` ) tells React to run this effect **only once** — **after the component's initial render**. This is similar to `componentDidMount` in class components.

**So yes:**

The API call inside will run **only once per mount**.

---

### ⚠ Important nuances

#### 1. Runs once *per mount*, not once forever

If the component unmounts and mounts again (e.g., navigating away and back), the effect will run again.

#### 2. Strict Mode (React 18+) runs effects twice in development

React's StrictMode intentionally runs effects twice in *development only* to detect unsafe side effects.

So you might see:

```
sql
```

[Copy code](#)

```
API call fired twice (DEV MODE only)
```

But in production, it will run only once.

---

## ✓ Summary

| Mode                        | With [] dependencies            |
|-----------------------------|---------------------------------|
| Development (StrictMode on) | Runs <b>twice</b> (intentional) |
| Production                  | Runs <b>once</b>                |
| If component remounts       | Runs again                      |

If you'd like, I can show you how to avoid double API calls in development or show best practices for fetching data in React.



Than means If the component re-renders, then it will make the API call again?

No — a re-render does **NOT** trigger the effect again when the dependency array is empty.

### ✓ With `useEffect(() => {...}, [])`:

- Initial mount? → effect runs ✓
- Re-render due to state/prop change? → effect does **not** run ✗
- Re-render for any other reason? → still does **not** run ✗
- Component unmount + re-mount? → effect runs again ✓

So the only times the API call runs are:

1. The component mounts for the first time
2. Any time the component mounts again later (e.g., navigating away and back, conditional rendering)

## Example

jsx

Copy code

```
useEffect(() => {  
  console.log("API call");  
}, []);
```

```
}, []);
```

- State update? → no new API call
- Parent re-render? → no new API call
- Props update? → no new API call

Only a *mount* triggers it.

---

### Why the confusion?

React **re-renders often**, but it does **not** re-run effects unless dependencies change or the component mounts.

---

If you want, I can show a visual diagram or code snippet to demonstrate the behavior.



How will I know if the component is unmounted?

You can detect when a component **unmounts** by returning a *cleanup function* inside `useEffect` .

### ✔ React calls the cleanup function right before the component unmounts

Example:

jsx

Copy code

```
useEffect(() => {  
  console.log("Component mounted");  
  
  return () => {  
    console.log("Component unmounted");  
  };  
}, []);
```

### How this works:

- When the component **mounts**, React runs the effect → "Component mounted"

- When the component is **about to unmount**, React runs the cleanup → "Component unmounted"

---

### ✅ Where this is useful:

- Canceling API requests
  - Clearing intervals/timeouts
  - Removing event listeners
  - Debugging mount/unmount behavior
- 

## Example with cleanup

jsx

Copy code

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Running...");  
  }, 1000);  
  
  return () => {  
    clearInterval(timer);  
    console.log("Timer cleared – component unmounted");  
  };  
}, []);
```

---

If you want, I can show you how to track mount/unmount with a custom hook (like `useIsMounted` ).



I actually did not understand the concept of mount and unmount

No problem — this is a core React concept, and once it clicks, everything about `useEffect` makes more sense.

Let me explain it **very simply**, with visuals and examples.

---



## What does “mount” mean in React?

► **Mount** = The first time a component appears on the screen

For example, you render a component:

jsx

Copy code

```
{show && <MyComponent />}
```

When `show` becomes `true`, React creates `MyComponent` and puts it into the DOM. This is called **mounting**.



React runs:

- the component function
- `useEffect(..., [])`
- layout/paint
- etc.



## What does “unmount” mean?

► **Unmount** = Removing the component from the screen

If `show` becomes `false`, React *removes* `<MyComponent />` from the DOM.

This is called **unmounting**.



React:

- removes the component
- cleans up memory
- calls the **cleanup function** from your `useEffect`



## Simple Example: Toggle a Component

Imagine this:

jsx

Copy code

```
function App() {  
  const [show, setShow] = useState(true);  
  
  return (  
    <div>  
      <button onClick={() => setShow(!show)}>Toggle</button>  
      {show && <Child />}  
    </div>  
  );  
}
```

## What happens?

| Action                                   | Result                            |
|------------------------------------------|-----------------------------------|
| App first renders                        | <Child /> is <b>mounted</b>       |
| Click "Toggle" → show becomes false      | <Child /> is <b>unmounted</b>     |
| Click "Toggle" again → show becomes true | <Child /> is <b>mounted again</b> |



## What React does behind the scenes

### When a component mounts

- React creates it for the first time.
- Runs the component function.
- Runs `useEffect(() => { ... }, [])`.

### When a component unmounts

React removes it completely, then runs the *cleanup function*:

jsx

Copy code

```
useEffect(() => {  
  console.log("mounted");  
  
  return () => {  
    console.log("unmounted");  
  };  
}, []);
```

```
} , [] ];
```

---

## Why is this important?

Because `useEffect` with empty dependency array behaves like this:

- ✓ Runs once → when component mounts
- ✗ Does NOT run again on re-render
- ✓ Runs cleanup → when the component unmounts

And if the component mounts again (because you show it again), the effect runs again.

---

## Easy real-world analogy

Think of a component like a **pop-up modal**.

- When the modal opens → **mounted**
- When the modal closes → **unmounted**

Every time you reopen it, React creates it again → **another mount**

---

## Visual Explanation

sql

Copy code

User loads page

↓

React creates <Component> → MOUNT

↓

User toggles UI, component removed → UNMOUNT

↓

User toggles UI, component shows again → MOUNT

---

If you want, I can illustrate this with a live code example or show how dev tools help visualize mounting and unmounting.



So, basically unmount means the component is no longer in the DOM?

Exactly! 

That's the simplest and most accurate way to think about it.

## **Unmount = The component is removed from the DOM**

When a component **unmounts**, React does two things:

1. **Removes its HTML/DOM elements from the page**
2. **Runs cleanup functions** in your `useEffect` hooks

So after unmounting:

- The component is gone from the UI
- Its state is gone
- Its event listeners are gone
- Its effects are cleaned up

---

## **Quick example**

jsx

Copy code

```
{show && <MyComponent />}
```

**When** `show = true`

- `<MyComponent />` is in the DOM
- It is **mounted**

**When** `show = false`

- `<MyComponent />` is removed from the DOM
- It is **unmounted**

## ✓ A simple DOM analogy

Imagine your component renders:

html

Copy code

```
<div>Hello</div>
```

When unmounted, React removes that entire `<div>` from the DOM — as if it never existed.

## ✓ Summary

| Term      | Meaning                                          |
|-----------|--------------------------------------------------|
| Mount     | Component added to the DOM for the first time    |
| Unmount   | Component removed from the DOM completely        |
| Re-render | Component stays in the DOM, but React updates it |

If you'd like, I can show you a small sandbox example where you can visually see components mounting and unmounting.



what is the cleanup function in useEffect?

The **cleanup function** in `useEffect` is a function you *return* from inside the effect. React runs it **when the component is about to unmount** or **before the effect runs again**.

## ✓ What the cleanup function looks like

jsx

Copy code

```
useEffect(() => {  
  console.log("Effect ran");  
})
```

```
    console.log( "Effect ran" ),

    return () => {
      console.log("Cleanup ran");
    };
  }, []);
```

### In this example:

- "Effect ran" → runs when the component mounts
- "Cleanup ran" → runs when the component unmounts

## 💡 Why does React need a cleanup function?

Effects often set up things that need to be torn down later, such as:

- event listeners
- intervals / timeouts
- subscriptions
- WebSocket connections
- API request cancellation

If you don't clean these up, they can keep running even after the component is gone → memory leaks, unexpected behavior, duplicate events, etc.

## 🧠 Common examples of cleanup

### 1 Clearing intervals/timeouts

jsx

Copy code

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("tick");
  }, 1000);

  return () => clearInterval(id); // cleanup
}, []);
```

### 2 Removing event listeners

jsx

Copy code

```
useEffect(() => {
  function handleResize() {
    console.log("resized");
  }

  window.addEventListener("resize", handleResize);

  return () => window.removeEventListener("resize", handleResize);
}, []);
```

### 3 Canceling fetch requests (AbortController)

jsx

Copy code

```
useEffect(() => {
  const controller = new AbortController();

  fetch("/api/data", { signal: controller.signal });

  return () => controller.abort(); // cleanup
}, []);
```

---

## When does cleanup run?

Depends on the dependencies:

- ✓ If dependency array is empty ( `[]` )
  - Cleanup runs only on unmount
- ✓ If there are dependencies ( `[value]` )

Cleanup runs in two cases:

1. Before the effect runs again (when `value` changes)
2. When the component unmounts

---

## Quick mental model

Effect runs → sets something up

## Cleanup runs → tears that thing down

React does this to keep everything tidy.

---

If you'd like, I can show you the execution order of effects step-by-step with a timeline.



```
useEffect(() => {  
  const unsubscribe = onAuthStateChanged(auth, (user)  
=> {  
    if (user) {  
      const { uid, email, displayName, photoURL } = user;  
      dispatch(  
        addUser({  
          uid: uid,  
          email: email,  
          displayName: displayName,  
          photoURL: photoURL,  
        })  
      );  
      navigate("/browse");  
    } else {  
      dispatch(removeUser());  
      navigate("/");  
    }  
  });  
  // Unsubscribe when component unmounts  
  return () => unsubscribe();  
}, []);
```

This was taken from Firebase user management API. Can you tell me why it was necessary to unsubscribe when the component unmounts?

Great question — this is **exactly** the kind of situation where cleanup in `useEffect` is essential.

You're using Firebase's `onAuthStateChanged` listener:

js

Copy code

```
const unsubscribe = onAuthStateChanged(auth, (user) => {  
  ...  
});
```

This registers a **persistent listener** that stays active until you manually remove it.

---

## ✅ Why do you need to unsubscribe on component unmount?

Because `onAuthStateChanged` keeps listening for auth state changes **even if your component is no longer on the screen**.

If you do NOT unsubscribe:

### ❌ 1. The listener keeps running after the component unmounts

The callback will still trigger whenever the authentication state changes — but your component is gone.

### ❌ 2. It may try to call `dispatch()` or `navigate()` after the component is removed

This can lead to:

- memory leaks
- "Can't perform a React state update on an unmounted component" warnings
- navigation errors
- duplicated listeners firing multiple times

### ❌ 3. You could accidentally register multiple listeners

Every time the component mounts, you add *another* listener if you don't clean up the old one.

This leads to:

- duplicate calls to `dispatch`
- duplicate redirects
- performance problems

- inconsistent state

---

## ✓ What does `unsubscribe()` do?

Firebase gives you this:

js

Copy code

```
const unsubscribe = onAuthStateChanged(auth, callback);
```

The returned `unsubscribe` function **removes the listener**.

You call it on cleanup:

js

Copy code

```
return () => unsubscribe();
```

This keeps everything tidy.

---

## 📌 Here's what React does internally

**On mount:**

- React runs your effect → listener is registered.

**On unmount:**

- React runs the cleanup → `unsubscribe()` removes the listener.

---

## 🧠 Simple visualization

**Without cleanup:**

powershell

Copy code

Mount → listener added

Unmount → listener stays alive! ❌

Mount again → second listener added

Unmount → second listener stays alive! ❌

Listeners pile up → messy, buggy behavior.

### With cleanup:

```
powershell
```

[Copy code](#)

Mount → listener added

Unmount → listener removed ✓

Mount again → listener added ✓

Unmount → listener removed ✓

Everything stays clean and predictable.



## Summary

You unsubscribe because:

| Reason                              | Why it matters                               |
|-------------------------------------|----------------------------------------------|
| Prevent memory leaks                | Listener running when not needed             |
| Avoid duplicate listeners           | Prevents multiple redirects, dispatches      |
| Avoid calling state after unmount   | Prevents React warnings and crashes          |
| Good practice with any subscription | Consistent with React's effect cleanup model |

---

If you'd like, I can explain what happens in your app **if you remove the unsubscribe** — with a real-world scenario using Firebase.

